



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Dokumentacja techniczna

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 11 czerwca 2026

Spis treści

1	Wprowadzenie	2
2	Architektura	2
3	Języki wejściowe — składnia plikowa	2
4	Przebieg rozwiązywania	3
5	Przykłady z obliczeniami	5
5.1	Prosty efekt — inercja i minimalizacja	5
5.2	Rosyjska ruletka — niedeterminizm przez releases	5
5.3	Konflikt w akcji złożonej — dekompozycje	6
5.4	Ramifikacja — skutek uboczny przez always	6
5.5	Niewykonalność — impossible i kwerenda executable	6
6	Testy	7

1 Wprowadzenie

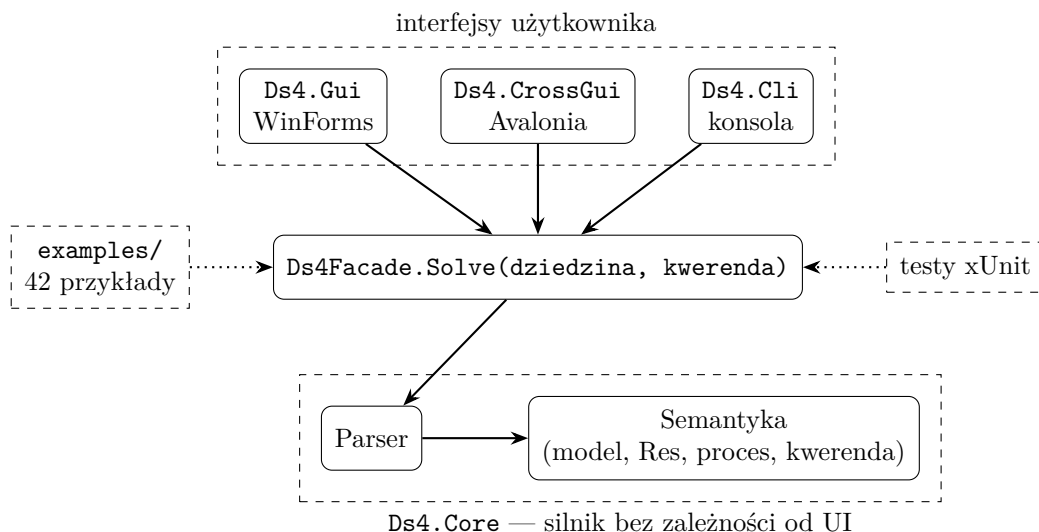
Aplikacja jest solverem dla klasy *DS4* systemów dynamicznych. Na wejściu przyjmuje tekstowy opis dziedziny (w języku akcji) oraz jedną kwerendę (w języku kwerend), buduje w pamięci model dziedziny i odpowiada **TAK** lub **NIE** na pytania Q1 (wykonywalność procesu) i Q2 (osiągnięcie celu), w wariantach *zawsze* i *kiedykolwiek*. Oprócz odpowiedzi program pokazuje krótkie uzasadnienie oraz pełne drzewo wykonań procesu (trace).

Definicje obu języków oraz ich semantyka znajdują się w *dokumentacji teoretycznej*; niniejszy dokument ich nie powtarza — opisuje wyłącznie budowę programu, przyjmowaną składnię plikową, przebieg obliczeń i testy. Tam, gdzie mowa o pojęciach formalnych (Σ , Σ_0 , Res, New, konflikty, dekompozycje), podajemy odsyłacze do odpowiednich sekcji dokumentacji teoretycznej (numeracja według jej wersji 2).

Implementacja: C# 12 / .NET 8. Silnik nie korzysta z żadnych bibliotek zewnętrznych; interfejsy graficzne używają WinForms (Windows) i Avalonii (wieloplatformowo). Testy — xUnit.

2 Architektura

System składa się z jednego silnika (*Ds4.Core*) i trzech wymiennych interfejsów. Całość spina jedna fasada.



- **Interfejsy** (WinForms / Avalonia / CLI) — tylko pola tekstowe, lista wbudowanych przykładów i przycisk „Oblicz”. Trzy frontendy korzystają z tej samej logiki.
- **Ds4Facade** — jedyne API używane przez interfejsy. Przyjmuje dwa napisy (dziedzina, kwerenda), zwraca odpowiedź TAK/NIE, uzasadnienie, rozmiary $|\Sigma|$ i $|\Sigma_0|$ oraz trace.
- **Ds4.Core** — czysty silnik: parser zamienia tekst na struktury danych, moduł semantyki buduje model i ewaluje kwerendę. Wymiana frontendu nie zmienia ani jednej linii silnika.
- **examples/** — 42 pary plików `.domain` + `.query`, ładowane do listy w GUI i CLI.
- **Testy** — xUnit; testują fasadę i moduły semantyki, w tym wszystkie przykłady względem plików `.expected`.

3 Języki wejściowe — składnia plikowa

Parser przyjmuje zapis ASCII konstrukcji zdefiniowanych w dokumentacji teoretycznej (sekcje 2.4 i 3.11). Poniższa tabela zestawia notację teoretyczną z zapisem plikowym.

Konstrukcja	Notacja teoretyczna	Zapis w pliku
efekt akcji	$A \text{ causes } \alpha \text{ if } \pi$	shoot causes !alive if loaded
zwolnienie inercji	$A \text{ releases } f \text{ if } \pi$	spin releases loaded if true
niewykonalność	impossible $A \text{ if } \pi$	impossible load if loaded
ograniczenie	always α	always p -> q
stan początkowy	initially α	initially !p and !q
fluent nieinercyjny	noninertial f	noninertial f
zdanie wartościujące	$\alpha \text{ after } \mathbb{A}_1; \dots; \mathbb{A}_n$	p after a; {b,c}
wariant obserwowalny	observable $\alpha \text{ after } \dots$	observable p after a

Formuły zapisuje się operatorami ! ($\neg$), & lub **and** ($\wedge$), | lub **or** ($\vee$), -> ($\rightarrow$), <-> ($\leftrightarrow$) oraz stałymi **true/false**. Komentarze zaczynają się od #. Deklaracje **fluents ...** i **actions ...** są opcjonalne — parser sam zbiera symbole występujące w zdaniach.

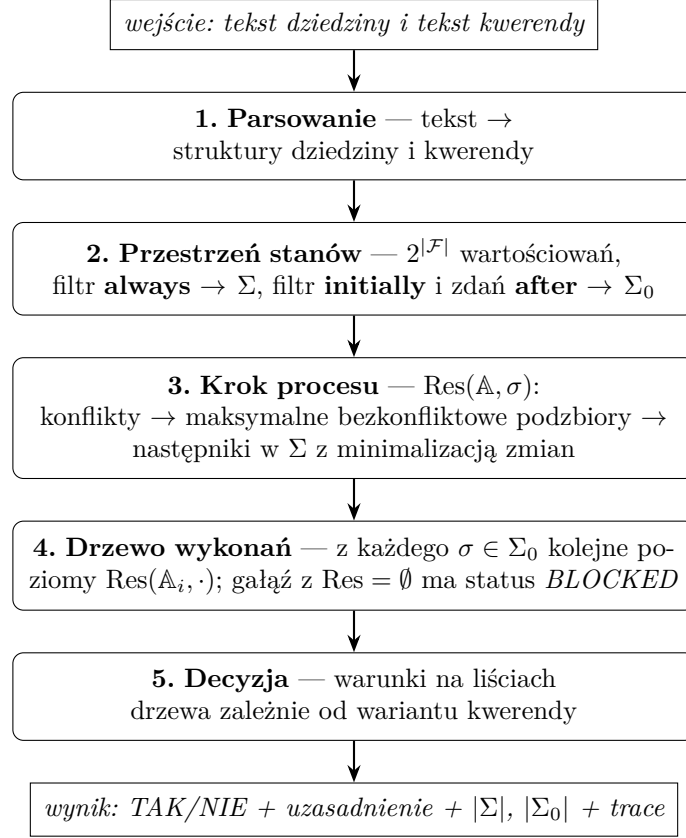
Zgodnie z dokumentacją teoretyczną **impossible** i **initially** są skrótami (odpowiednio: $A \text{ causes } \perp \text{ if } \pi$ oraz $\alpha \text{ after } \varepsilon$) — parser rozpoznaje je jako osobne słowa kluczowe, ale semantycznie traktuje tak samo jak rozwinięcia.

Kwerendy. Plik kwerendy zawiera jedno zdanie. Akcję złożoną zapisuje się w nawiasach klamrowych ($\{a, b\}$; nawiasy można pominąć dla akcji jednoelementowej), proces — jako ciąg akcji złożonych rozdzielonych średnikami. Cztery warianty pytań:

Pytanie	Wariant	Zapis w pliku
Q1: wykonywalność	zawsze	necessary executable after a; {b,c}
	kiedykolwiek	possibly executable after a; {b,c}
Q2: cel γ	zawsze	necessary !alive after spin; shoot
	kiedykolwiek	possibly !alive after spin; shoot

4 Przebieg rozwiązywania

Obliczenie przebiega w pięciu krokach. Program realizuje wprost definicje z dokumentacji teoretycznej — dla małych dziedzin (kilka–kilkanaście fluentów) pełne przeszukiwanie $2^{|\mathcal{F}|}$ stanów jest szybkie, a wierność definicjom była ważniejsza niż optymalizacje.



Krótko o każdym kroku (w nawiasach sekcje dokumentacji teoretycznej z definicjami):

1. **Parsowanie.** Z tekstu powstaje lista fluentów, akcji, reguł **causes** / **releases** / **impossible**, ograniczeń **always** / **initially** oraz proces i cel kwerendy.
2. **Przestrzeń stanów.** Generujemy wszystkie wartościowania fluentów i odsiewamy niezgodne z **always** — zostaje Σ (sekcja 2.5.1). Z Σ wybieramy stany zgodne z **initially** i ze zdaniem **after** — zostaje Σ_0 (sekcja 2.5.2), czyli zbiór możliwych stanów początkowych przy opisie częściowym.
3. **Krok procesu.** Dla akcji prostej: jeśli aktywna jest reguła **impossible**, następników brak; w przeciwnym razie następnikami są stany z Σ spełniające aktywne efekty i minimalizujące zbiór zmian New (sekcje 2.5.3–2.5.6) — tak realizujemy inercję (Z1) i skutki uboczne ograniczeń (Z6). Fluent zwolniony regułą **releases** zawsze wnosi swój literal do New, dzięki czemu oba jego wartościowania są nieporównywalne i oba przeżywają minimalizację (niedeterminizm, Z3). Dla akcji złożonej: wykrywamy konflikty par akcji w stanie σ (sekcja 3.3), wyznaczamy maksymalne bezkonfliktowe podzbiory — dekompozycje (sekcja 3.4) — i jako $\text{Res}(A, \sigma)$ bierzemy sumę wyników po dekompozycjach (sekcja 3.5).
4. **Drzewo wykonań.** Z każdego stanu początkowego rozwijamy drzewo: dzieci korzenia to $\text{Res}(A_1, \sigma)$, wnuki — $\text{Res}(A_2, \cdot)$ itd. (sekcja 3.8–3.9). Gałąź ucięta przed końcem procesu jest *zablokowana*; gałąź pełnej długości kończy się *liściem końcowym*.
5. **Decyzja** (sekcja 3.12). Kwantyfikator po stanach początkowych jest zawsze uniwersalny: przy opisie częściowym prawdziwy stan początkowy nie jest znany, więc odpowiedź musi uwzględniać każdy stan zgodny z **initially**. Warunki (we wszystkich przypadkach wymagamy $\Sigma_0 \neq \emptyset$):
 - **possibly executable** — z *każdego* stanu początkowego istnieje liść końcowy;

- **possibly** γ — z *każdego* stanu początkowego istnieje liść końcowy spełniający γ ;
- **necessary executable** — brak gałęzi zablokowanych i istnieją liście końcowe;
- **necessary** γ — jak wyżej oraz *wszystkie* liście końcowe spełniają γ .

5 Przykłady z obliczeniami

Pięć przykładów z wbudowanego zestawu; każdy pokazuje inny mechanizm. Zapisy dziedzin i kwerend są dosłownie tym, co przyjmuje program.

5.1 Prosty efekt — inercja i minimalizacja

```
initially !p
make causes p if true
```

Kwerenda: `necessary p after make`

- $\mathcal{F} = \{p\}$, brak **always**, więc $\Sigma = \{\{p\}, \{\neg p\}\}$; **initially** `!p` daje $\Sigma_0 = \{\{\neg p\}\}$.
- $\text{Res}(\text{make}, \neg p)$: aktywny efekt p ; kandydaci spełniający p : tylko $\{p\}$, $\text{New} = \{p\}$ — wynik $\{\{p\}\}$.
- Jedyne liść końcowy spełnia p , gałęzi zablokowanych brak. **Odpowiedź: TAK.**

5.2 Rosyjska ruletka — niedeterminizm przez releases

```
fluents loaded, alive
actions spin, shoot, load
initially alive
spin releases loaded if true
shoot causes !alive if loaded
impossible load if loaded
load causes loaded if !loaded
```

Kwerendy: `possibly !alive after spin; shoot` oraz `necessary !alive after spin; shoot`

- $|\Sigma| = 4$; **initially** `alive` ogranicza tylko *alive*, więc Σ_0 ma dwa stany: $\{l, a\}$ i $\{\neg l, a\}$ (opis częściowy).
- $\text{Res}(\text{spin}, \sigma)$: brak efektów **causes**, zwolniony *loaded* — oba wartościowania *loaded* przeżywają minimalizację, więc z każdego stanu początkowego dostajemy $\{l, a\}$ i $\{\neg l, a\}$.
- $\text{Res}(\text{shoot}, \cdot)$: z $\{l, a\}$ aktywny efekt $\neg \text{alive}$ daje $\{l, \neg a\}$; z $\{\neg l, a\}$ żadna reguła nie jest aktywna — inercja zostawia $\{\neg l, a\}$.
- Z każdego z dwóch stanów początkowych istnieje więc ścieżka kończąca się $\{l, \neg a\} \models \neg \text{alive}$ — ale wśród liści jest też $\{\neg l, a\}$, więc nie wszystkie ścieżki kończą się celem. **Odpowiedzi: possibly — TAK, necessary — NIE.**

5.3 Konflikt w akcji złożonej — dekompozycje

```
initially !p
makeP causes p if true
clearP causes !p if true
```

Kwerenda: necessary p after {makeP,clearP}

- $\Sigma = \{\{p\}, \{\neg p\}\}$, $\Sigma_0 = \{\{\neg p\}\}$.
- W stanie $\neg p$ akcje *makeP* i *clearP* wymuszają literały przeciwne (p i $\neg p$) — konflikt. Maksymalne bezkonfliktowe podzbiory: $\{makeP\}$ i $\{clearP\}$.
- $\text{Res}(\{makeP, clearP\}, \neg p) = \text{Res}(makeP, \neg p) \cup \text{Res}(clearP, \neg p) = \{\{p\}, \{\neg p\}\}$.
- Liść $\{\neg p\}$ nie spełnia p . **Odpowiedź: NIE.**

5.4 Ramifikacja — skutek uboczny przez always

```
always p -> q
initially !p and !q
makeP causes p if true
```

Kwerenda: necessary q after makeP

- Z czterech wartościowań ograniczenie *always p -> q* odrzuca $\{p, \neg q\}$, więc $|\Sigma| = 3$; $\Sigma_0 = \{\{\neg p, \neg q\}\}$.
- $\text{Res}(makeP, \{\neg p, \neg q\})$: aktywny efekt p ; jedynym stanem w Σ spełniającym p jest $\{p, q\}$ — fluent q zmienia się jako skutek uboczny, bo stan $\{p, \neg q\}$ nie należy do Σ .
- Jedyne liść spełnia q . **Odpowiedź: TAK.**

5.5 Niewykonalność — impossible i kwerenda executable

```
fluents loaded
actions load
initially loaded
impossible load if loaded
load causes loaded if !loaded
```

Kwerenda: possibly executable after load

- $\Sigma = \{\{l\}, \{\neg l\}\}$, $\Sigma_0 = \{\{l\}\}$.
- $\text{Res}(load, \{l\})$: aktywna jest reguła **impossible**, więc następników brak — jedyna gałąź drzewa jest zablokowana już na pierwszym kroku.
- Ze stanu początkowego nie istnieje żadna pełna ścieżka. **Odpowiedź: NIE.**

6 Testy

Zestaw testów (xUnit, projekt `Ds4.Tests`) obejmuje:

- **testy jednostkowe** — parser formuł i obu języków, generacja Σ i Σ_0 , Res dla akcji prostych (efekty, zwolnienia, niewykonalność, minimalizacja), wykrywanie konfliktów i dekompozycje, budowa drzewa wykonań i ewaluacja kwerend;
- **testy integracyjne** — każdy z 42 wbudowanych przykładów (24 z odpowiedzią TAK, 18 z NIE) jest rozwiązywany przez fasadę, a wynik porównywany z plikiem `.expected`;
- **testy regresyjne** — przypadki błędów znalezionych w trakcie rozwoju (m.in. warunkowe `causes`).

Uruchomienie: `dotnet test` w katalogu `app/` (na Windows także skrypt `RUN_TESTS.bat`).

Przypisanie testów do członków zespołu znajduje się w osobnym dokumencie „Wkład osobowy”.